INDEX

NAME: Manjusri.N STD.: _____ SEC.: _____ ROLL NO.: _____ SUB.: Computer Network

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.	22/7/26	Basic Networking Commands	2	
2.	23/7/26	Study of different types of Netbook	6	
3.	30/7/26	Learning & Assignment of IP address manually	10	
4.	30/7/26	Implement packet sniffing using raw socket in python	14	o/p verified
5.	6/8/26	Error detection and correction	16	
6.	1/8/26	Develop a customised ping command to test the server community	20	
6.	13/8/26	Building an anonymous FTP Scanner using python	22	
7.	10/9/26	Develop a Simple calculator using XML-RPC	26	
8.	17/9/26	Develop a program to create reverse shell using TCP Socket	30	o/p verified

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
9.	24/9/26	Design a Simple techno & configure 1 router, 2 switches pcs using cisco packet	32	
10.	26/9/26	Customize switch with network modules using cisco packet tracer	36	
11.	26/9/26	Examine network address using cisco packet tracer	40	
12.	26/9/26	Nmap to Discover live hosts using Nmap on tryhackme platform	42	
13.	1/10/26	Demonstrate network forensics using the PCAP - X-ray tool	44	
14.	3/10/26	Capture, Save & analyze network on TCP/UDP/IP/HTTP/ARP/ DNS/ICMP using wireshark tool	46	
15.	8/10/26	Analyze different types of Server using Nmap tool	48.	

completed

8/10/26

Exp no: 1
22/1/26

BASIC NETWORKING COMMANDS

Aim:
To study & execute basic networking commands in windows & linux

WINDOWS COMMANDS:

1. ipconfig:
   Display IP address Subnet mask, dyault gateway
2. ipconfig /all:
   Shows detailed network information
3. ping google.com:
   Test network connectivity
4. Tracert google.com:
   Trace route to destination
5. nslookup google.com:
   Display DNS information
6. route print:
   Display routing table
7. netstat:
   Display active connection
8. netstat -a:
   Display network statistics
9. getmac:
   Display MAC address
10. hostname:
    Check hostname
11. arp -a:
    [illegible] table

Output:

1. IPV4 address 192.168.1.20
   Subnet Mask . . . . . : 255.255.255.0
   Default Gateway . . . : 192.168.1.1

2. Reply from 8.8.8.8
   Packet: Sent - 4
   Recieved - 4
   Lost - 0

11. arp -a:

Internet address	Physical address	Type
192.168.1.1	AA-BB-CC-DD	Dynamic
192.168.1.105	11-22-33-44	Dynamic

4. 1. 192.168.1.1
   2. ISP Router
   3. Google network

5. Name: google.com
   Address: 142.250.xxx.xxx

6. route print:
   IPV4 route

Network destination	Netmask	Interface
0.0.0.0	0.0.0.0	192.168.1.100
192.168.1.0	255.255.255.0	192.168.1.100

7. netstat:

Proto	Local address	Foreign address	State
TCP	192.168.1.100:5000	142.250.190.78:443	Established

8. netstat -a:

Proto	Local address	Foreign address	State
TCP	0.0.0.0:135	0.0.0.0:0	Listening
TCP	192.168.1.100:5000	142.250.190:443	Established
UDP	0.0.0.0:1900	242:0.0.423	

9. Physical Address Transport name
   00-1A-2B-3C-4D \Device\Tcpip_{123456}

10. hostname:
    DESKTOP-AB123CD

LINUX NETWORKING COMMANDS:

1. ip addr / ifconfig:
   Displays IP address
2. ip route:
   Display routing table
3. hostname:
   Displays hostname
4. ping google.com:
   Test connectivity
5. Traceroute google.com /:
   Trace network route
6. nslookup google.com /:
   sudo apt install
   DNS lookup
7. netstat -a / ss -a:
   Display network statistics
8. arp -a / ip neigh
   Display ARP table
9. ip link / ifconfig / link /ether:
   MAC address
10. cat /etc / resolv.conf:
    DNS configuration
11. ip link show:
    check interface status
12. ss -tuln:
    Display listening ports
13. curl google.com / wget google.com:
    Check internet connectivity

Result:
Thus the commands are done successfully in Linux

Output:

1. 2: eth0
   inet 192.168.1.100/24
2. default via 192.168.1.1 dev eth0
   192.168.1.0/24 dev eth0
3. Desktop - PC
4. 64 bytes from google.com : time = 20 ms
5. 1 192.168.1.1
   2 10.10.0.1
   3 google.com
6. Server : 8.8.8.8
   Name : google.com
   Addr : 142.250.183.14
7. tcp 0 0 localhost:22 LISTEN
   tcp 0 0 192.168.1.100:5000 ESTABLISHED
8. ? (192.168.1.1) at aa:bb:dd:ee:ff
9. link/ether 08:00:27:12:34:56
10. name server 8.8.8.8
17. eth0 : UP
    lo : UP
12. tcp LISTEN 0.0.0.0:22
    tcp LISTEN 0.0.0.0:80
13. <!DOCTYPE HTML>
    <Html>
    google

Pno:2
17b6

STUDY OF DIFFERENT TYPES OF NETWORK CABLES

Aim:

Study of different types of Network cables

a) Understand different types of Network cables:

Different types of cables used in networking are:

1. Unshielded twisted pair cable
2. Shielded Twisted pair cable
3. Coaxial Cable
4. Fibre optic cable

Cable type	category	Maximum data transmission	Advantages/ Disadvantages	Application/ use
UTP	Category 3	10 bps	Adv: * cheaper in cost * Easy to install Dis: * More prone to EMI * Electromagnetic interface and noise	10 base-T ethernet
	category 5	upto to 100 Mbps		Fast Ethernet Gigabit Ethernet
	categorys5e	1 Gbps		
STP	category 6, 6a	10 Gbps	Adv: * Shielded * Faster than UTP * less to noise and interference Dis: * Expensive * Greater installation effort * Hig	Gigabit Ethernet 10G Ethernet (55m) widely used in data center
SSTP	category 7	10 Gbps		
coaxial cable	RG-6 RG-59 RG-11	10-100 Mbps	* High bandwidth * Immune to interference * Low Loss bandwidth * Versatile Dis: * Limited distance * cost * Size is bulky	Speed of signal is 500 m TV network internet connection

cable type	category	Maximum Data Transmission	Advantages/ Disadvantages	Application/ use
fibre optics cables	Single mode Multi mode	100 Gbps	Adv: * High Speed * High bandwidth * long distance Dis: * Expensive * Requires skilled installer	Maximum distance of fibre around 100 meters

Result:

Thus different types of cables are studied successfully. The cables functioned correctly & communication b/w the connected device was established

Expno: 3
29/7/26

LEARNING & ASSIGNMENT OF IP ADDRESS MANUALLY TO COMPUTERS

Aim:

To understood IP addressing scheme and manually assign static IP address to computers on network

Requirements:

Two or max PC, network switch / hub, Ethernet cables (RJ45), windows / Linux OS.

Theory:

* IP address is a unique numerical label assigned to each device connected to a computer network.
* IPV4 address are 32-bit number written in dotted decimal address
* Classes of IPV4: class A (1.0.0.0 - 126.255.255.255)
* Subnet mask determines the network & host portions default gateway is routers ID DNS Server translates domain names to IPs
* Manual IP assignment is used for severs & Network devices requiring a pamanent address. DHCP assign IP s dynamically

Procedure:

1. In windows:

Open network & sharing enter → changes Adapter settings → Right click local area connection → properties → IPV4 → setup manually

Code:

sudo ip addr add 192.168.1.10/24 dev eth0

sudo ip route add default via 192.168.1.1

ip a show eth0

ping 192.168.1.1

nmcli con mod 'wired connection 1' ipv4.addresses 192.168.1.10/24

nmcli con mod 'wired connection 1' ipv4.method manual

nmcli con mod 'wired connection 1'

2. Enter IP address, default Gateway
3. Enter DNS Server : 8.8.8.8, click OK
4. In Linux: Use command 'sudo IP addr add 192.168.1.10/24 div eth0
5. To set gateway in Linux : "sudo ip route add default via 192.168.1.1.
6. For persistent configuration in Linux, edit/etc/network interface or use nmcli
7. Verify the configuration using ipconfig or ipa
8. ping default gateway confirm network connectivity
9. Ping another computer on same subnet to verify communication

Result:
Static IP address were successfully assigned to computer. Connectivity was verified using ping commands b/w hosts on same subnet.

Expt no: 4
18/26

IMPLEMENT PACKET SNIFFING USING RAW POCKETS IN PYTHON

Aim:

To implement a packet sniffer using raw socket in python that captures live network traffic & parses ethernet IP & transport headers

Procedure:

Step 1: Set up environment & confirm python 3 in installed

Step 2: Understand packets structure you will parse

Step 3: Create raw socket, create a file sniffer.py & start

Step 4: parse ethernet header, add func to unpack first 14 bytes

Step 5: parse the IP header add

Step 6: parse TCP / UDP / ICMP headers

Step 7: write main sniffing loop

Step 8: Run the Sniffer

Step 9: Stop the Sniffer with Ctrl + C

```
import socket
import struct
# Create RAW Socket
sniffer = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPRO
# Local machine IP
host = socket.gethostbyname(socket.gethostname())
sniffer.bind((host, 0))
# Include IP headers
sniffer.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)
# Enable promiscuous mode (Windows)
sniffer.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)

print("Packet Sniffer Started...\n")
try:
    while True:
        raw_data, addr = sniffer.recvfrom(65535)
        ip_header = raw_data[0:20]
        iph = struct.unpack("!BBHHHBBH4s4s", ip_header)

        version = iph[0] >> 4
        ihl = iph[0] & 0x0F
        ttl = iph[5]
        protocol = iph[6]
        source = socket.inet_ntoa(iph[8])
        destination = socket.inet_ntoa(iph[9])

        print("-----------------------------------------------")
        print("Version       :", version)
        print("Header Length:", ihl * 4, "bytes")
        print("TTL           :", ttl)
        print("Protocol      :", protocol)
        print("Source IP     :", source)
        print("Destination   :", destination)
        print("Packet Length:", len(raw_data))

except KeyboardInterrupt:
    # Disable promiscuous mode before exiting
    sniffer.ioctl(socket.SIO_RCVALL, socket.RCVALL_OFF)
    print("\nStopped")
```

Result:

Hence, the implementation of packet Sniffer has been Verified & Successful.

```
----------------------------------------
Version      : 4
Header Length: 20 bytes
TTL          : 62
Protocol     : 17
Source IP    : 142.251.222.206
Destination  : 172.16.10.48
Packet Length: 150
----------------------------------------
Version      : 4
Header Length: 20 bytes
TTL          : 62
Protocol     : 17
Source IP    : 142.251.222.206
Destination  : 172.16.10.48
Packet Length: 53
----------------------------------------
Version      : 4
Header Length: 20 bytes
TTL          : 62
Protocol     : 17
Source IP    : 142.251.222.206
Destination  : 172.16.10.48
Packet Length: 53
----------------------------------------
Version      : 4
Header Length: 20 bytes
TTL          : 128
Protocol     : 17
Source IP    : 172.16.10.48
Destination  : 142.251.222.206
Packet Length: 63
----------------------------------------
Version      : 4
Header Length: 20 bytes
TTL          : 62
Protocol     : 17
Stopped
```

Exp no: 5
1/8/26

DEVELOP A CUSTOMISED PING COMMAND TO TEST THE SERVER CONNECTIVITY

Aim:

To develop a customized ping command in python that ~~connectivi~~ check the connectivity between a client and a server by sending ICMP Echo Request packets & displaying the respone time and status

Procedure:

1. Install python and VS code
2. Create a python file named ping.py
3. Import the required module
4. Accept the hostname or IP address from the user
5. Detect the operating System
6. Accept using -n 4 on windows or ping -c 4 on Linux/mac OS
7. Execute the ping command using the subprocess module
8. Display the ping output, including reply time and packet statistics
9. Check the return code to determine whether the server is reachable
10. Display reachable or Unreachable based on the result.

to develop a c
that ~~connects~~
a client and
Request packets
& status

```
import platform

print("=" * 50)
print("Customized Ping Utility")
print("=" * 50)

host = input("Enter Website or IP Address: ")

system = platform.system()

if system == "Windows":
    command = ["ping", "-n", "4", host]
else:
    command = ["ping", "-c", "4", host]

try:
    result = subprocess.run(command, capture_output=True, text=True)

    print("\nPing Results\n")
    print(result.stdout)

    if result.returncode == 0:
        print("Server Status : Reachable")
    else:
        print("Server Status : Unreachable")

except Exception as e:
    print("Error:", e)
```

```
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win3
Enter "help" below or click "Help" above for more information.

= RESTART: C:/Users/REC/AppData/Local/Programs/Python/Python313/server connectivity.py
================================================
Customized Ping Utility
================================================
Enter Website or IP Address: google.com

Ping Results


Pinging google.com [142.251.221.142] with 32 bytes of data:
Reply from 142.251.221.142: bytes=32 time=50ms TTL=120
Reply from 142.251.221.142: bytes=32 time=57ms TTL=120
Reply from 142.251.221.142: bytes=32 time=78ms TTL=120
Reply from 142.251.221.142: bytes=32 time=63ms TTL=120

Ping statistics for 142.251.221.142:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 50ms, Maximum = 78ms, Average = 62ms

Server Status : Reachable
```

Result:

The network connectivity of the specified host was successfully tested, and the server status was displayed correctly.

EXPNO:6 BUILDING AN ANONYMOUS FTP SCANNER USING PYTHON

Aim:

To develop a python-based anonymous FTP scanner using the built-in ftplib module & determine whether an FTP server permits anonymous access.

Procedure:

1. Import the required FTP and all_errors classes from python ftplib module
2. Define a function anonymous_ftp_scan() that accepts the FTP server address and port number
3. Create an FTP client object using FTP()
4. Connect to the specified FTP server using ftp.connect()
5. Display the FTP server's welcome message
6. Attempt anonymous authentication using
   * username
   * password
7. If authentication succeeds, display message indicating that anonymous access is enabled
8. Use ftp.nlst() to retrieve the accessible directory listing
9. Display the files & directories available to the anonymous user
10. If the connection or fails, catch the FTP related & display the error

```
from ftplib import FTP, all_errors
def anonymous_ftp_scan(host, port=21):
    print("=" * 50)
    print(" ANONYMOUS FTP SCANNER")
    print("=" * 50)
    print(f"Target: {host}")
    print(f"Port  : {port}")
    print()
    ftp = FTP()
    try:
        print("[1] Connecting to FTP server...")
        ftp.connect(host, port, timeout=10)
        print("[+] FTP server is reachable")
        print("[+] Server response:", ftp.getwelcome())
        print("[+] Server response:", ftp.getwelcome())
        print("\n[2] Testing anonymous login...")
        ftp.login(user="anonymous", passwd="anonymous@example.com")
        print("[+] Anonymous login SUCCESSFUL")
        print("[!] The FTP server permits anonymous access.")
        print("\n[3] Checking accessible directory...")
        files = ftp.nlst()
        if files:
            print("[+] Directory listing is available:")
            for item in files:
                print("    ", item)
        else:
            print("[-] Directory is empty or listing is unavailable.")
        ftp.quit()
    except all_errors as error:
        print("[-] Anonymous login failed or FTP connection failed.")
        print("[!] Error:", error)
        try:
            ftp.close()
        except Exception:
            pass
target = input("Enter FTP server address: ").strip()
anonymous_ftp_scan(target)
```

```
Target: ftp.example.com
Port  : 21

[1] Connecting to FTP server...
[-] Anonymous login failed or FTP connection failed.
[!] Error: [Errno 11001] getaddrinfo failed

========== RESTART: C:/Users/HDC0222058/Documents/CN EXP-6.py ==========
Enter FTP server address:  192.168.1.10
==================================================
 ANONYMOUS FTP SCANNER
==================================================
Target: 192.168.1.10
Port  : 21

[1] Connecting to FTP server...
[-] Anonymous login failed or FTP connection failed.
[!] Error: timed out

========== RESTART: C:/Users/HDC0222058/Documents/CN EXP-6.py ==
Enter FTP server address: 127.0.0.1
==================================================
 ANONYMOUS FTP SCANNER
==================================================
Target: 127.0.0.1
Port  : 21

[1] Connecting to FTP server...
[-] Anonymous login failed or FTP connection failed.
[!] Error: [WinError 10061] No connection could be made because the target machine actively refused it
```

11. Close the FTP connection using ftp.quit() or ftp.close()

12. Execute the program by accepting the authorized FTP server address from the user

Result:

The program was successfully executed to test an FTP server for anonymous access

Develop A Simple Calculator Using XML-RPC

Expno:7

Aim:

To develop a simple calculator using the XML-RPC protocol in python to perform basic arithmetic operation between a client & a server

Procedure:

1. Create a python file named Server.py and write the XML-RPC calculator server code
2. Run Server.py to start the XML-RPC server on port 8000
3. Create another python file named client.py and write the client code.
4. Run client.py while server in running
5. Enter the first and second number
6. Select operation:
   * Addition
   * Subtraction
   * Multiplication
   * Division
7. client sends the request to the Server using XML-RPC
8. The Server performs the selected operation & results the result
9. The client display the result

```
from xmlrpc.server import SimpleXMLRPCServer
server = SimpleXMLRPCServer(("localhost", 8000))
print("XML-RPC Calculator Server Running on Port 8000...")
def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
def multiply(a, b):
    return a * b
def divide(a, b):
    if b == 0:
        return "Division by zero is not allowed."
    return a / b
server.register_function(add, "add")
server.register_function(subtract, "subtract")
server.register_function(multiply, "multiply")
server.register_function(divide, "divide")
server.serve_forever()
```

```
import xmlrpc.client
server = xmlrpc.client.ServerProxy("http://localhost:8000")
a = int(input("Enter First Number: "))
b = int(input("Enter Second Number: "))
print("\nSelect Operation")
print("1. Addition")
print("2. Subtraction")
print("3. Multiplication")
print("4. Division")
choice = input("Enter your choice (1-4): ")
if choice == '1':
    print("Result =", server.add(a, b))
elif choice == '2':
    print("Result =", server.subtract(a, b))
elif choice == '3':
    print("Result =", server.multiply(a, b))
elif choice == '4':
    print("Result =", server.divide(a, b))
else:
    print("Invalid Choice")
```

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64
bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

= RESTART: C:/Users/HDC0222021/AppData/Local/Programs/Python/Python313/server
1.py
XML-RPC Calculator Server Running on Port 8000...

= RESTART: C:/Users/HDC0222021/AppData/Local/Programs/Python/Python313/client
1.py
Enter First Number: 65
Enter Second Number: 32

Select Operation
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter your choice (1-4): 2
Result = 33
```

Result:

The XML-RPC calculator was successfully implemented using python. The client communication with Server & perform addition Subtraction, multiplication, & division Succesfull

Expno:8
17/9/24

DEVELOP A PROGRAM TO CREATE REVERSE SHELL USING TCP SOCKET

```
C:\Users\HDC0222007\Downloads\Experiment 8>python server.py
Waiting for client...
Connected by: ('127.0.0.1', 65500)
Client: helloo!!!
Enter reply: Hiii!!!
```

```
C:\Users\HDC0222007>cd "C:\Users\HDC0222007\Downloads\Experiment 8"

C:\Users\HDC0222007\Downloads\Experiment 8>python client.py
Connected to server
Enter message: helloo!!!
Server: Hiii!!!
Enter message:
```

server.py client.py

C: > Users > HDC0222007 > Downloads > Experiment 8 > server.py > ...

```python
import socket
SERVER_IP = "127.0.0.1"
PORT = 5000
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((SERVER_IP, PORT))
server.listen(1)
print("Waiting for client...")
conn, addr = server.accept()
print("Connected by:", addr)
while True:
    message = conn.recv(1024).decode()
    if not message or message.lower() == "exit":
        break
    print("Client:", message)
    response = input("Enter reply: ")
    conn.send(response.encode())
conn.close()
server.close()
print("Connection closed")
```

Aim:

To develop a simple TCP socket based client server program that demonstrates communicati between a client and server using python sockets.

Procedure:

1. Import the socket module in python
2. Define the server IP address as 127.0.0.1 & post number as 5001
3. Create a TCP socket using socket.AF_INET and socket.SOCK_STREAM
4. Connect the client socket using client.connect()
5. Display a message indicating that the connect is established
6. Get message from the user using input()
7. Send the message using client.send()
8. Receive the server response using & display it
9. Repeat the process until the user enters
10. Close the connection using client.close()

```
C:\Users\HDC0222007\Downloads\Experiment 8>python server.py
Waiting for client...
Connected by: ('127.0.0.1', 65500)
Client: helloo!!!
Enter reply: Hiii!!!
```

```
C:\Users\HDC0222007>cd "C:\Users\HDC0222007\Downloads\Experiment 8"

C:\Users\HDC0222007\Downloads\Experiment 8>python client.py
Connected to server
Enter message: helloo!!!
Server: Hiii!!!
Enter message:
```

server.py client.py

C: > Users > HDC0222007 > Downloads > Experiment 8 > server.py > ...

```python
import socket
SERVER_IP = "127.0.0.1"
PORT = 5000
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((SERVER_IP, PORT))
server.listen(1)
print("Waiting for client...")
conn, addr = server.accept()
print("Connected by:", addr)
while True:
    message = conn.recv(1024).decode()
    if not message or message.lower() == "exit":
        break
    print("Client:", message)
    response = input("Enter reply: ")
    conn.send(response.encode())
conn.close()
server.close()
print("Connection closed")
```

Expno:8
17/9/24

DEVELOP A PROGRAM TO CREATE REVERSE SHELL USING TCP SOCKET

Aim:

To develop a simple TCP socket based client server program that demonstrates communicati between a client and server using python sockets.

Procedure:

1. Import the socket module in python
2. Define the server IP address as 127.0.0.1 & post number as 5001
3. Create a TCP socket using socket.AF_INET and socket.SOCK_STREAM
4. Connect the client socket using client.connect()
5. Display a message indicating that the connect is established
6. Get message from the user using input()
7. Send the message using client.send()
8. Receive the server response using & display it
9. Repeat the process until the user enters
10. Close the connection using client.close()

server.py client.py

C: > Users > YHDC0222007 > Downloads > Experiment 8 > client.py > ...

```python
import socket
SERVER_IP = "127.0.0.1"
PORT = 5000
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((SERVER_IP, PORT))
print("Connected to server")
while True:
    message = input("Enter message: ")
    if message.lower() == "exit":
        break
    client.send(message.encode())
    response = client.recv(1024).decode()
    print("Server:", response)
client.close()
print("Connection closed")
```

Result:

The python program for TCP socket communication was executed to establish a client - server connection, send & receive.

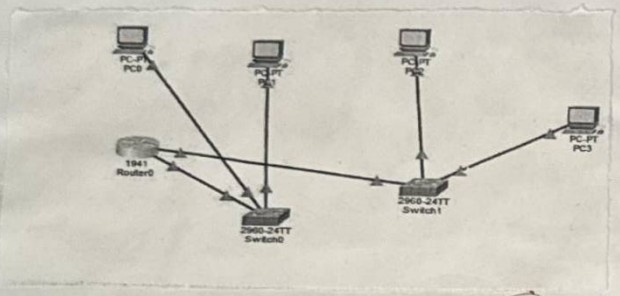
Expt:9 DESIGN A SIMPLE TOPOLOGY AND CONFIGURE ONE ROUTER TWO SWITCHES, AND PCS USING CISCO PACKET TRACER

Aim:

To design and configure a simple network topology consisting of one router, two switches & 4 PCs using cisco packet tracer.

Procedure:

1. Open cisco packet tracer
2. Add 1 router, 2 switches & 4 PCs to the workspace
3. Connect the device using copper straight-through cables
4. Open the router CLI and configure the 0/0 interface with IP addr 192.168.1.1 & Subnet mask 255.255.255.0. Enable the
5. Assign IP addr, subnet masks & default gateway to all PCs
6. Save the router configuration.
7. Test connectivity between PCs using ping command
8. Verify successful communication b/w all devices




```
Router>enable
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#interface gigabitEthernet0/0
Router(config-if)#ip address 192.168.1.1 255.255.255.0
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up
exit
Router(config)#interface gigabitEthernet0/1
Router(config-if)#ip address 192.168.2.1 255.255.255.0
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up
exit
Router(config)#
Router#
%SYS-5-CONFIG_I: Configured from console by console

Router#
Router#exit
```

```
Cisco Packet Tracer PC Command Line 1.0
C:\>ping 192.168.1.3

Pinging 192.168.1.3 with 32 bytes of data:

Reply from 192.168.1.3: bytes=32 time=13ms TTL=128
Reply from 192.168.1.3: bytes=32 time=1ms TTL=128
Reply from 192.168.1.3: bytes=32 time<1ms TTL=128
Reply from 192.168.1.3: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.1.3:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 13ms, Average = 3ms

C:\>ping 192.168.2.2

Pinging 192.168.2.2 with 32 bytes of data:

Request timed out.
Reply from 192.168.2.2: bytes=32 time<1ms TTL=127
Reply from 192.168.2.2: bytes=32 time<1ms TTL=127
Reply from 192.168.2.2: bytes=32 time<1ms TTL=127

Ping statistics for 192.168.2.2:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

```
C:\>
C:\>ping 192.168.2.2

Pinging 192.168.2.2 with 32 bytes of data:

Reply from 192.168.2.2: bytes=32 time=9ms TTL=127
Reply from 192.168.2.2: bytes=32 time<1ms TTL=127
Reply from 192.168.2.2: bytes=32 time<1ms TTL=127
Reply from 192.168.2.2: bytes=32 time=11ms TTL=127

Ping statistics for 192.168.2.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 11ms, Average = 5ms

C:\>ping 192.168.2.3

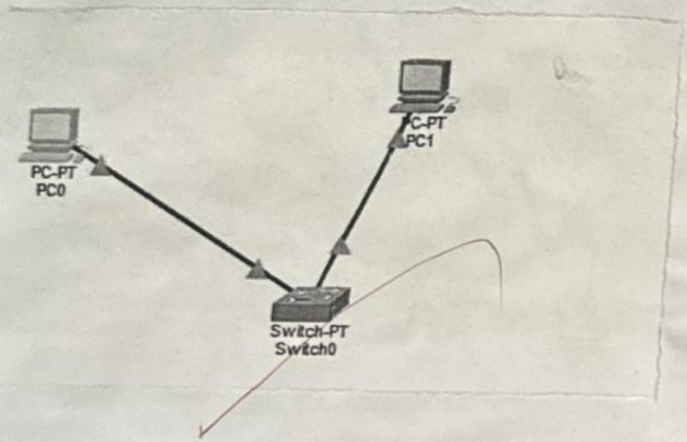
Pinging 192.168.2.3 with 32 bytes of data:

Reply from 192.168.2.3: bytes=32 time<1ms TTL=127
Reply from 192.168.2.3: bytes=32 time<1ms TTL=127
Reply from 192.168.2.3: bytes=32 time<1ms TTL=127
Reply from 192.168.2.3: bytes=32 time<1ms TTL=127

Ping statistics for 192.168.2.3:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

Result:

The network topology was successfully designed and configured in cisco packet tracer.



Expno:10

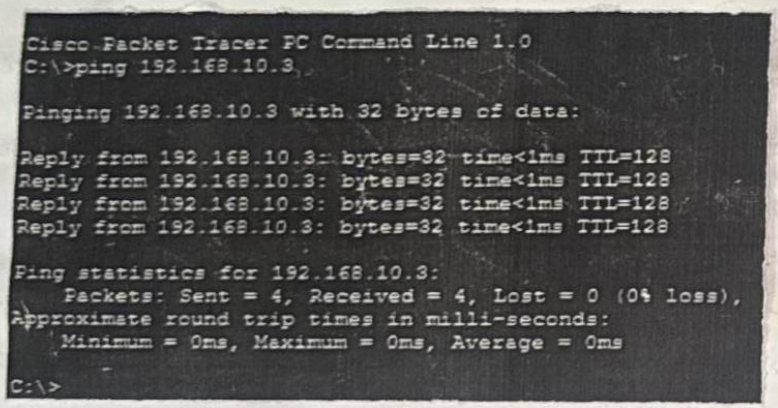
USING CISCO PACKET TRACER

Aim:

To customize a cisco switch by adding or removing network modules using cisco packet tracer & verify the changes in the switch interfaces

Procedure:

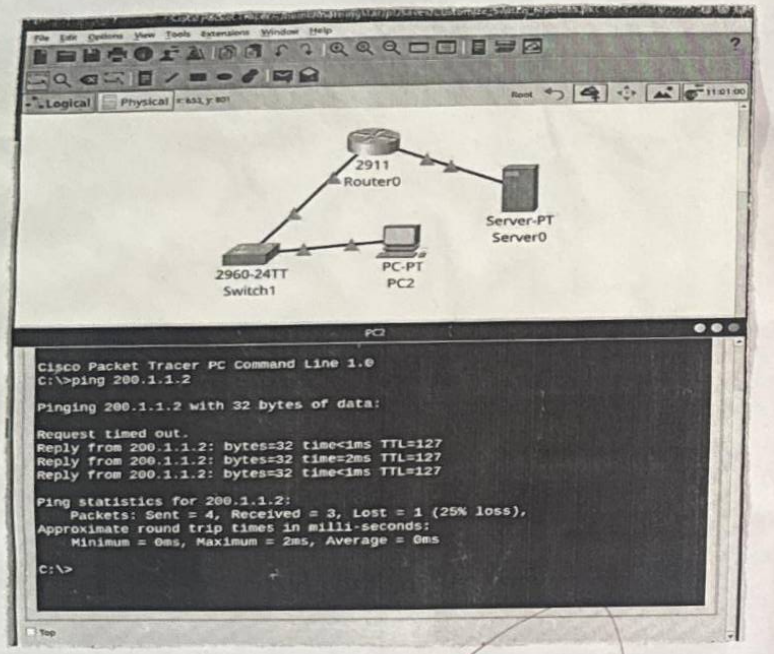
1. Open cisco packet tracer & add a modular switch
2. Open the physical tab & turn off the switch
3. Remove an existing module if required and insert a compatible network module.
4. Turn on the switch & verify the new interfaces.
5. Connect two PCs to the switch using copper straight through cables
6. Configure the PCs with IP address 192.168.10.2 & 192.168.10.3 using subnet mask 255.255.255.0
7. Test connectivity using ping command
8. Save the packet file.



Result:
The switch was customized by adding a compatible network module, and the new interface were verified.

Expno: 11

EXAMINE NETWORK ADDRESS TRANSLATION USING CISCO PACKET



Aim:

To configure and examine NAT using cisco packet tracer and verify communication blw private network & public network

Procedure:

1. Open cisco packet tracer & add one router, one switch, one PC, & one Server
2. Connect the device using copper straight through cables
3. Configure the router inside interface with appropriate IP address & enable NAT
4. Configure static NAT to map private IP 192.168.1.2 to public IP 200.1.1.100.
5. Assign IP address & default gateways to the PC & Server
6. Test connectivity using Ping command
7. Verify NAT translations using show ip nat translations & view statistics
8. Save the router configuration.

Result:

NAT was configured using cisco packet tracer by mapping the private IP addr. 192.168.1.2 to the public IP addr 200.1.1.100.

```
root@ip-10-48-165-144:~/Documents# nmap -sn -PR 10.48.165.144/24
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-10-12 15:23 UTC
Nmap scan report for ip-10-48-165-46.ap-south-1.compute.internal (10.48.165.46)
Host is up (0.00029s latency).
MAC Address: 02:FF:E6:D9:3C:9F (Unknown)
Nmap scan report for ip-10-48-165-61.ap-south-1.compute.internal (10.48.165.61)
Host is up (0.00023s latency).
MAC Address: 02:FF:F8:31:E2:75 (Unknown)
Nmap scan report for ip-10-48-165-93.ap-south-1.compute.internal (10.48.165.93)
Host is up (0.00019s latency).
MAC Address: 02:FF:CD:46:17:79 (Unknown)
Nmap scan report for ip-10-48-165-97.ap-south-1.compute.internal (10.48.165.97)
Host is up (0.00028s latency).
MAC Address: 02:FF:EA:89:DB:47 (Unknown)
Nmap scan report for ip-10-48-165-105.ap-south-1.compute.internal (10.48.165.105)
Host is up (0.00021s latency).
MAC Address: 02:FF:E9:2E:DF:B3 (Unknown)
Nmap scan report for ip-10-48-165-150.ap-south-1.compute.internal (10.48.165.150)
Host is up (0.00027s latency).
MAC Address: 02:FF:D7:B6:13:3B (Unknown)
Nmap scan report for ip-10-48-165-200.ap-south-1.compute.internal (10.48.165.200)
Host is up (0.00032s latency).
MAC Address: 02:FF:DD:F3:44:2D (Unknown)
Nmap scan report for ip-10-48-165-237.ap-south-1.compute.internal (10.48.165.237)
Host is up (0.00048s latency).
MAC Address: 02:FF:ED:D6:B5:39 (Unknown)
Nmap scan report for ip-10-48-165-254.ap-south-1.compute.internal (10.48.165.254)
Host is up (0.00019s latency).
MAC Address: 02:FF:C2:68:84:A5 (Unknown)
Nmap scan report for ip-10-48-165-144.ap-south-1.compute.internal (10.48.165.144)
Host is up.
Nmap done: 256 IP addresses (10 hosts up) scanned in 2.17 seconds
root@ip-10-48-165-144:~/Documents#
```

```
root@ip-10-48-165-144:~/Documents# # ICMP Echo discovery
root@ip-10-48-165-144:~/Documents# sudo nmap -PE -sn 10.48.165.0/24
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-10-12 15:38 UTC
Nmap scan report for ip-10-48-165-61.ap-south-1.compute.internal (10.48.165.61)
Host is up (0.000041s latency).
MAC Address: 02:FF:F8:31:E2:75 (Unknown)
Nmap scan report for ip-10-48-165-93.ap-south-1.compute.internal (10.48.165.93)
Host is up (0.000057s latency).
MAC Address: 02:FF:CD:46:17:79 (Unknown)
Nmap scan report for ip-10-48-165-97.ap-south-1.compute.internal (10.48.165.97)
Host is up (0.000040s latency).
MAC Address: 02:FF:EA:89:DB:47 (Unknown)
Nmap scan report for ip-10-48-165-105.ap-south-1.compute.internal (10.48.165.105)
Host is up (0.000041s latency).
MAC Address: 02:FF:E9:2E:DF:B3 (Unknown)
Nmap scan report for ip-10-48-165-150.ap-south-1.compute.internal (10.48.165.150)
Host is up (0.000035s latency).
MAC Address: 02:FF:D7:B6:13:3B (Unknown)
Nmap scan report for ip-10-48-165-200.ap-south-1.compute.internal (10.48.165.200)
Host is up (0.000041s latency).
MAC Address: 02:FF:DD:F3:44:2D (Unknown)
Nmap scan report for ip-10-48-165-215.ap-south-1.compute.internal (10.48.165.215)
Host is up (0.000054s latency).
MAC Address: 02:FF:D0:B1:42:2B (Unknown)
Nmap scan report for ip-10-48-165-237.ap-south-1.compute.internal (10.48.165.237)
Host is up (0.000028s latency).
MAC Address: 02:FF:ED:D6:B5:39 (Unknown)
Nmap scan report for ip-10-48-165-254.ap-south-1.compute.internal (10.48.165.254)
Host is up (0.000065s latency).
MAC Address: 02:FF:C2:68:84:A5 (Unknown)
Nmap scan report for ip-10-48-165-144.ap-south-1.compute.internal (10.48.165.144)
Host is up.
Nmap done: 256 IP addresses (10 hosts up) scanned in 3.71 seconds
root@ip-10-48-165-144:~/Documents#
```

Expno: 12

TRY HACK ME PLATFORM

Aim:

To discover live hosts in a network using Nmap by performing:

* ARP Scan
* ICMP ping scan
* TCP ping scan
* UDP ping scan

Procedure:

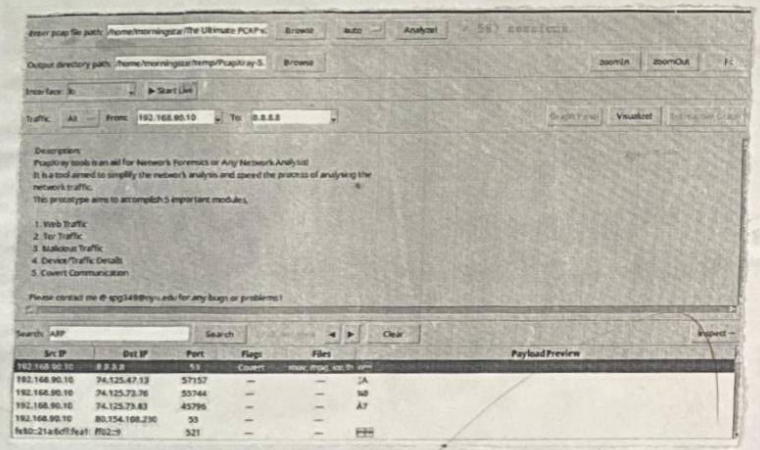
1. Log in to the tryhackme platform & open the required Nmap room.
2. Deploy the target machine and note its IP address.
3. Start the attack box and open the terminal
4. Perform ARP scan using nmap -PR <target IP>
5. perform ICMP ping scan using nmap -sn -PE < target - IP>
6. perform TCP ping scan using nmap -sn -PS80 < target - IP>
7. perform a UDP ping scan nmap -sn -PU53 < target - IP >
8. Observe the output to identify live hosts & verify their availability.

```
root@ip-10-48-165-144:~/Documents# # UDP Ping
root@ip-10-48-165-144:~/Documents# sudo nmap -PU -sn 10.48.165.0/24
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-10-12 15:41 UTC
Nmap scan report for ip-10-48-165-61.ap-south-1.compute.internal (10.48.165.61)
Host is up (0.00010s latency).
MAC Address: 02:FF:F8:31:E2:75 (Unknown)
Nmap scan report for ip-10-48-165-93.ap-south-1.compute.internal (10.48.165.93)
Host is up (0.000032s latency).
MAC Address: 02:FF:CD:46:17:79 (Unknown)
Nmap scan report for ip-10-48-165-97.ap-south-1.compute.internal (10.48.165.97)
Host is up (0.000072s latency).
MAC Address: 02:FF:EA:B0:DB:47 (Unknown)
Nmap scan report for ip-10-48-165-105.ap-south-1.compute.internal (10.48.165.105)
Host is up (0.000036s latency).
MAC Address: 02:FF:19:2E:DF:B3 (Unknown)
Nmap scan report for ip-10-48-165-150.ap-south-1.compute.internal (10.48.165.150)
Host is up (0.000051s latency).
MAC Address: 02:FF:D7:B6:13:3B (Unknown)
Nmap scan report for ip-10-48-165-200.ap-south-1.compute.internal (10.48.165.200)
Host is up (0.00011s latency).
MAC Address: 02:FF:DD:F3:44:2D (Unknown)
Nmap scan report for ip-10-48-165-215.ap-south-1.compute.internal (10.48.165.215)
Host is up (0.000061s latency).
MAC Address: 02:FF:D0:B1:42:2B (Unknown)
Nmap scan report for ip-10-48-165-237.ap-south-1.compute.internal (10.48.165.237)
Host is up (0.000044s latency).
MAC Address: 02:FF:ED:D6:B5:39 (Unknown)
Nmap scan report for ip-10-48-165-254.ap-south-1.compute.internal (10.48.165.254)
Host is up (0.000048s latency).
MAC Address: 02:FF:C2:68:B4:A5 (Unknown)
Nmap scan report for ip-10-48-165-144.ap-south-1.compute.internal (10.48.165.144)
Host is up.
Nmap done: 256 IP addresses (10 hosts up) scanned in 2.50 seconds
root@ip-10-48-165-144:~/Documents#
```

```
root@ip-10-48-165-144:~/Documents# # TCP SYN Ping (default port 80)
root@ip-10-48-165-144:~/Documents# sudo nmap -PS -sn 10.48.165.0/24
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-10-12 15:41 UTC
Nmap scan report for ip-10-48-165-61.ap-south-1.compute.internal (10.48.165.61)
Host is up (0.00007s latency).
MAC Address: 02:FF:F8:31:E2:75 (Unknown)
Nmap scan report for ip-10-48-165-93.ap-south-1.compute.internal (10.48.165.93)
Host is up (0.00004s latency).
MAC Address: 02:FF:CD:46:17:79 (Unknown)
Nmap scan report for ip-10-48-165-97.ap-south-1.compute.internal (10.48.165.97)
Host is up (0.00002s latency).
MAC Address: 02:FF:EA:B0:DB:47 (Unknown)
Nmap scan report for ip-10-48-165-105.ap-south-1.compute.internal (10.48.165.105)
Host is up (0.00005s latency).
MAC Address: 02:FF:19:2E:DF:B3 (Unknown)
Nmap scan report for ip-10-48-165-150.ap-south-1.compute.internal (10.48.165.150)
Host is up (0.00007s latency).
MAC Address: 02:FF:D7:B6:13:3B (Unknown)
Nmap scan report for ip-10-48-165-200.ap-south-1.compute.internal (10.48.165.200)
Host is up (0.00002s latency).
MAC Address: 02:FF:DD:F3:44:2D (Unknown)
Nmap scan report for ip-10-48-165-215.ap-south-1.compute.internal (10.48.165.215)
Host is up (0.00007s latency).
MAC Address: 02:FF:D0:B1:42:2B (Unknown)
Nmap scan report for ip-10-48-165-237.ap-south-1.compute.internal (10.48.165.237)
Host is up (0.00005s latency).
MAC Address: 02:FF:ED:D6:B5:39 (Unknown)
Nmap scan report for ip-10-48-165-254.ap-south-1.compute.internal (10.48.165.254)
Host is up (0.00008s latency).
MAC Address: 02:FF:C2:68:B4:A5 (Unknown)
Nmap scan report for ip-10-48-165-144.ap-south-1.compute.internal (10.48.165.144)
Host is up.
Nmap done: 256 IP addresses (10 hosts up) scanned in 2.49 seconds
root@ip-10-48-165-144:~/Documents#
```

Result:

Live hosts discovery was performed using Nmap on try hack me platform through ARP, ICMP, TCP & UDP Scans.



Exp no: 13

DEMONSTRATE NETWORK FORENSICS USING PCAP XRAYTOOL

Aim:

To perform network forensics by analysing a PCAP file using PCAP X-ray tool to identify network protocols, suspicious activities communication pattern, transfered files & security indicators.

Procedure:

1. Open a web browser & access the PCAP X-ray tool
2. upload a sample PCAP file for analysis
3. View the dashboard & record packet counts, protocols, & source and distination IP addr
4. Analyze protocol distribution, including TCP, UDP, HTTPS, HTTP, DNS
5. Examine hosts & conversation to identify communication pattern, ports, & data transfer
6. Check for plaintext credentials, if available
7. Inspects transfered file & identify suspicious network activities
8. Generate & Save the analysis report in HTML.

```
~
→ tshark -r "$F" -Y "dns.flags.response==0" -T fields -e ip.src -e dns.qry.name -e dn
s.qry.type | sort | uniq -c | sort -rn | head -20
     53 172.16.81.99    v10.events.data.microsoft.com   28
     44 172.16.81.99    v10.events.data.microsoft.com   1
     25        iTunes_Ctrl_73F532408528F239._dacp._tcp.local   255
     25 192.168.7.26    iTunes_Ctrl_73F532408528F239._dacp._tcp.local   255
     24 172.16.81.99    settings-win.data.microsoft.com 28
     21 172.16.81.99    wpad.ad2024.weberlab.de 1
     20 172.16.81.99    wpad.ad2024.weberlab.de 28
     18 172.16.81.99    settings-win.data.microsoft.com 1
     18 172.16.81.99    cp601.prod.do.dsp.mp.microsoft.com      28
     17 172.16.81.99    3.tlu.dl.delivery.mp.microsoft.com      28
     17 172.16.81.99    3.tlu.dl.delivery.mp.microsoft.com      1
     15 192.168.121.2   blog.webernetz.net      1
     15 192.168.110.9   blog.webernetz.net      1
     15 172.16.81.99    vm39-ad2024.ad2024.weberlab.de  1
     15 172.16.81.99    cp601.prod.do.dsp.mp.microsoft.com      1
     14        _homekit._tcp.local,_airplay._tcp.local,_sleep-proxy._udp.local12,12,
12
     14 85.215.94.29    router1.weberlab.de     1
     14 192.168.7.26    _homekit._tcp.local,_airplay._tcp.local,_sleep-proxy._udp.loc
al 12,12,12
     13 172.16.81.99    g.live.com      28
     12 172.16.81.99    g.live.com      1
```

Result:

Network forensics was performed using the PCAP-X-ray tools. The packet capture was analyzed to examine network protocols & security indicators.

Expt no:14 ON TCP/UDP/IP/HTTP/HTTPS/ICMP/DNS USING TOOLS

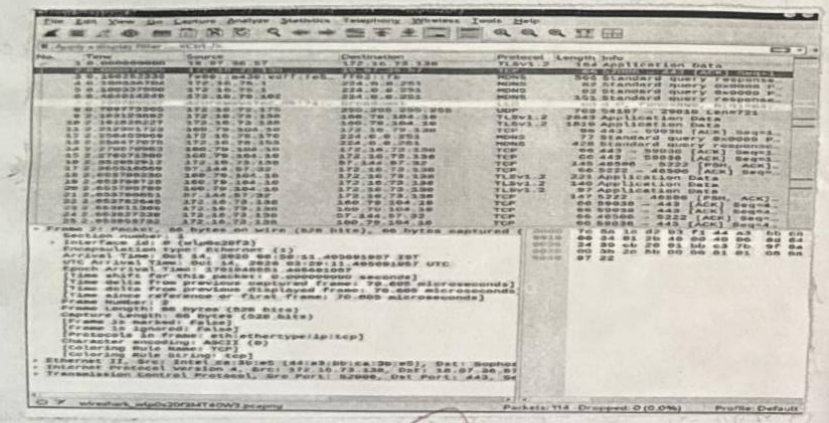


Aim:

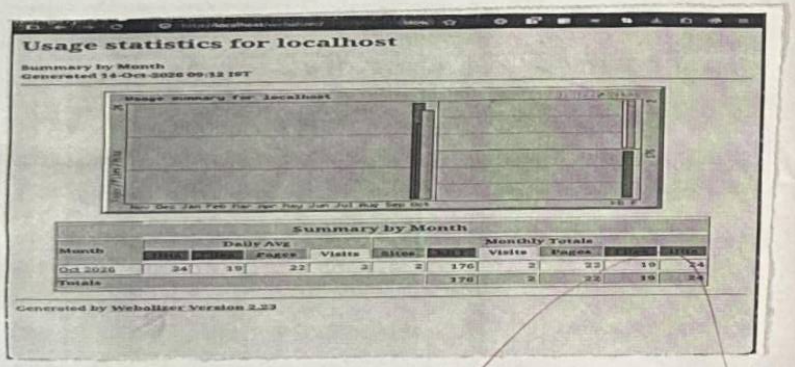
To capture, save, and analyze network traffic of TCP, UDP, IP, HTTP, ARP, DHCP, ICMP, & DNS protocols using wireshark network protocols analyzer.

Procedure:

1. Download & install wireshark with Npcap, if required
2. Open wireshark and select the active network interface, such as wifi
3. Start packet capture by double-clicking the selected interface
4. Generate network traffic using browsing ping command & DNS queries.
5. Apply display filters to analyze diffent protocols
6. Stop capture after collecting sufficient packets.
7. Save the captured traffic.
8. Examine packet details, source & destination address, protocols types & packet content.



Result:
Network traffic was captured, saved, & analyzed successfully using wireshark.



Expt 15

TO ANALYZE THE DIFFERENT TYPES OF SERVERS USING WEBALIZER TOOL

Aim:

To install, configure, & use webalizer tool to analyze web server log files & generate statistical reports for different types of web servers.

Procedure:

1. Install apache web server & the webalizer tools on ubuntu linux.
2. Configure webalizer to read the web server access log file
3. Start the web server and open the website in a browser
4. Generate web traffic by refreshing page
5. Run webalizer to process the access log file & reports
6. Open the generated HTML reports in a web browser
7. Analyze monthly, daily & hourly statistics, including hits, page view & bandwidth usage
8. Examine browser statistics, operating system, HTTP & top request page

Result:

Webalizer was used to analyze web server log files & generate statistical reports.

completed
18/10/24